RELATÓRIO TÉCNICO

Monitor de Ruído de Máquina com Detecção Preditiva de Falhas

BitDogLab Noise Monitor System

Projeto Final — EmbarcaTech
Plataforma: BitDogLab V7 (Raspberry Pi RP2040)
Linguagem: C | Abril de 2026

Klysmann G. F. Barbosa

Link do vídeo: videoEmbarcatech[1].mp4

a) Apresentação

O BitDogLab Noise Monitor System é um sistema embarcado de monitoramento preditivo que detecta automaticamente anomalias acústicas em máquinas rotativas (motores elétricos, bombas, compressores), acionando alarme quando o nível de ruído excede o limiar aprendido — indicando possíveis falhas iminentes como rolamento desgastado, desalinhamento de eixo ou cavitação.

O problema central abordado é a detecção precoce de falhas em máquinas industriais. Métodos tradicionais de manutenção corretiva geram custos elevados e paradas não planejadas. A manutenção preditiva baseada em análise de vibração exige sensores caros (acelerômetros). Uma alternativa de baixo custo é o monitoramento de ruído: máquinas saudáveis têm assinatura acústica estável; falhas incipientes alteram essa assinatura antes da quebra catastrófica.

Contexto de aplicação

- Industrial: motores elétricos, bombas centrífugas, compressores, exaustores
- Manutenção preditiva: detecção precoce de falhas em rolamentos
- Comissionamento: verificação de desalinhamento pós-manutenção

Motivação

Demonstrar conceitos avançados de sistemas embarcados — dual-core, aprendizado estatístico embarcado e monitoramento em tempo real — usando exclusivamente os componentes da BitDogLab V7, com destaque para o microfone analógico como sensor primário.

b) Objetivos

Objetivo Geral

Desenvolver um sistema embarcado no RP2040 que aprenda a assinatura acústica normal de uma máquina em operação e detecte anomalias de ruído que indiquem falhas iminentes, com baixa taxa de falsos positivos.

Objetivos Específicos

- Implementar algoritmo de aprendizado acústico com threshold adaptativo ($\mu + N \cdot \sigma$) e remoção de outliers
- Utilizar microfone analógico como sensor primário (único sensor necessário)
- Implementar dois níveis de ameaça graduados: PROBABLE (falha incipiente) e CONFIRMED (falha severa)
- Explorar a arquitetura dual-core do RP2040: Core 0 para segurança/detecção, Core 1 para interface
- Fornecer feedback visual (display OLED, LED RGB) e sonoro (buzzer PWM estéreo) em tempo real
- Permitir ajuste de sensibilidade (2σ, 3σ, 4σ) para adaptação a diferentes tipos de máquina

c) Requisitos Funcionais

ID	Descrição	Entrada	Saída
RF01	Aprender assinatura acústica (5 s, 100 Hz, 500 amostras, remoção 10% outliers)	Microfone ADC GPIO28	threshold = $\mu + N \cdot \sigma$
RF02	Detectar som anômalo acima do threshold aprendido	Nível de som atual (filtrado)	Boolean anômalo
RF03	Calcular nível de ameaça: PROBABLE (<50% excesso) ou CONFIRMED (≥50% excesso)	Razão som/threshold	ThreatLevel (0–3)
RF04	Ajustar sensibilidade via interface (Alta/2σ, Média/3σ, Baixa/4σ)	Joystick + Botão B	SensitivityIndex
RF05	Resposta graduada: LED e buzzer proporcionais ao nível de ameaça	ThreatLevel	PWM buzzer / LED RGB
RF06	Controle por botões: A=calibrar+armar, B=desarme (IRQ), SW=log UART	GPIO buttons	SystemMode
RF07	Display OLED com status em tempo real (30 Hz)	SystemState	Framebuffer I2C
RF08	Alternar entre tela principal e tela de status detalhado	Joystick + Botão B	ScreenMode
RF09	Histórico circular de leituras dos últimos 60 segundos	SensorData	Array[60]
RF10	Log UART sob demanda via botão SW (joystick)	GPIO22	Dados formatados

Restrições de operação

- Microfone calibrado com offset DC (500 amostras em silêncio no boot)
- Filtro de média móvel de 10 amostras no sinal do microfone
- Debounce de 2 segundos entre alertas para evitar disparos múltiplos
- Snooze automático de 5 minutos após alerta CONFIRMED
- Botão B usa interrupção GPIO (borda de descida) para desarme imediato
- Uso de RAM máximo: ~50 KB (264 KB disponíveis no RP2040)

d) Arquitetura de Hardware

O sistema utiliza exclusivamente componentes integrados na placa BitDogLab V7, baseada no microcontrolador Raspberry Pi RP2040.

Componente	Interface / Pino	Papel no Sistema
RP2040 (Dual-Core)	—	Core 0 = segurança/detecção (10 Hz), Core 1 = interface (30 Hz)

Microfone analógico	ADC2 / GPIO28	Sensor primário — 100 Hz, 12 bits, filtro média móvel 10 amostras
Sensor temp. interno	ADC4 (interno)	Sensor auxiliar — compensação térmica (planejado)
Display OLED SSD1306	I2C1 / GPIO14-15	Interface visual — 128×64 px, 400 kHz, 4 telas
Buzzer estéreo (A+B)	PWM / GPIO10-11	Feedback sonoro — padrões diferenciados por nível de ameaça
LED RGB	GPIO13/17/16	Indicador visual rápido de estado
Botão A	GPIO5	Calibrar ambiente e armar sistema
Botão B	GPIO6	Desarme de emergência via IRQ + combinação com joystick
Joystick SW	GPIO22	Log UART sob demanda
Joystick X/Y	ADC0/GPIO26, ADC3/GPIO29	Navegação da interface

Diagrama de Conexões (Pinout)

```
ADC:   GPIO26 (ADC0) → Joystick X
       GPIO28 (ADC2) → Microfone
       GPIO29 (ADC3) → Joystick Y
       ADC4 (interno) → Sensor Temperatura RP2040

I2C1:  GPIO14 (SDA)  → OLED SSD1306
       GPIO15 (SCL)  → OLED SSD1306

GPIO:  GPIO5  → Botão A
       GPIO6  → Botão B (IRQ borda de descida)
       GPIO22 → Joystick SW
       GPIO13 → LED R   GPIO17 → LED G   GPIO16 → LED B

PWM:   GPIO10 → Buzzer A
       GPIO11 → Buzzer B
```

e) Arquitetura do Firmware

Estrutura em Camadas

```
┌──────────────────────────────────────────────────────────┐
│ APPLICATION LAYER                                        │
│ Máquina de estados, gerenciamento de modos               │
│ e coordenação de eventos (main.c)                        │
├──────────────────────────────────────────────────────────┤
│ BUSINESS LOGIC LAYER                                     │
│ Aprendizado acústico (alarm.c)                           │
│ Detecção de anomalias + estatística robusta              │
├──────────────────────────────────────────────────────────┤
│ HARDWARE ABSTRACTION LAYER (HAL)                         │
│ sensors.c (microfone + temperatura)                      │
│ display.c (SSD1306 + UI)                                 │
├──────────────────────────────────────────────────────────┤
│ PICO SDK                                                 │
│ ADC, GPIO, PWM, I2C, PIO, DMA, mutex, multicore          │
└──────────────────────────────────────────────────────────┘
```

Arquitetura Dual-Core

```
┌──────────────────────────────────────────────────────────────────────┐
│ CORE 0 — Segurança (core0_security.c)                                │
│ • Leitura de sensores: 10 Hz (a cada 100 ms)                         │
│ • Detecção de anomalias: 2 Hz (a cada 500 ms)                        │
│ • Processa flag de IRQ (desarme BTN B)                               │
│ • Gerenciamento de estados: ARMED, TRIGGERED, SNOOZE                 │
│ • Auto-reset após 5 minutos em TRIGGERED                             │
├──────────────────────────────────────────────────────────────────────┤
│ CORE 1 — Interface (core1_interface.c)                               │
│ • Atualização do display OLED: 30 Hz (a cada 33 ms)                  │
│ • Leitura de joystick com confirmação (3 leituras)                   │
│ • Leitura de botões A e SW (polling via IRQ)                         │
│ • Controle de buzzer: 10 Hz  |  Controle de LED RGB                  │
├──────────────────────────────────────────────────────────────────────┤
│ COMUNICAÇÃO INTER-CORE                                               │
│ • SystemState compartilhada em SRAM                                  │
│ • mutex_t g_state_mutex protege todo acesso                          │
│ • mutex_t g_adc_mutex para acesso ao ADC                             │
│ • volatile flags para IRQ: g_disarm_requested, etc.                  │
└──────────────────────────────────────────────────────────────────────┘
```

Módulos Principais

Arquivo	Função / Loop	Responsabilidade
alarm.c	**alarm_learn_environment()**	Coleta 500 amostras a 100 Hz, ordena, remove outliers 10%, calcula μ, σ, threshold
alarm.c	**alarm_detect()**	Compara ruído atual com threshold, classifica ameaça por excesso percentual
sensors.c	**sensors_read_all()**	Leitura simultânea de microfone (offset DC + média móvel) e temperatura interna
display.c	**display_draw_screen()**	Renderiza tela ativa no framebuffer e envia via I2C ao SSD1306
core0_security.c	**Loop 10 Hz / 2 Hz**	Leitura de sensores, detecção de ameaças, gerenciamento de estado e IRQ
core1_interface.c	**Loop 30 Hz / 10 Hz**	Display, joystick (3 leituras confirmação), buzzer, LED RGB

alarm.c — Aprendizado Acústico e Detecção

Coleta 500 amostras do microfone a 100 Hz (5 segundos), ordena com qsort, descarta percentis inferiores e superiores (10% cada), calcula média robusta (μ) e desvio padrão (σ), e define threshold = μ + N·σ (N ajustável por sensibilidade). A detecção compara o ruído atual com o threshold e classifica a ameaça com base no percentual de excesso. Complexidade de ordenação: O(n log n).

sensors.c — Leitura de Sensores

Gerencia o microfone com calibração de offset DC (500 amostras), filtro de média móvel de 10 amostras e leitura da temperatura interna do RP2040 via ADC4. Mantém histórico circular de 60 segundos.

display.c — Driver SSD1306 e UI

Driver completo do SSD1306 com framebuffer de 1 KB, fonte 5×7 embutida e primitivas gráficas. Cinco telas implementadas:

- Main: barra de ruído, temperatura, sensibilidade

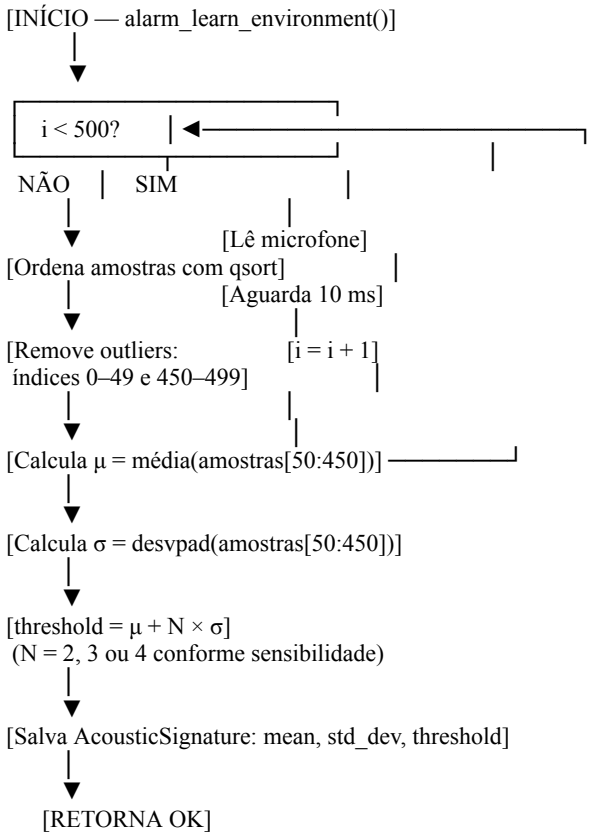
- Calibração (Learning): barra de progresso 0–100%
- Armada (Armed): barra de ruído com marcador de threshold
- Alerta (Alert): texto piscante com nível da ameaça
- Status: valores detalhados (μ, σ, threshold)

core1_interface.c — Interface com Joystick

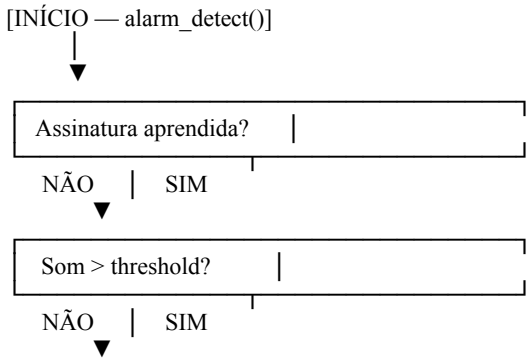
Implementa leitura do joystick com filtro de confirmação: requer 3 leituras consecutivas na mesma direção para confirmar movimento. Ações são executadas apenas quando o Botão B é pressionado simultaneamente.

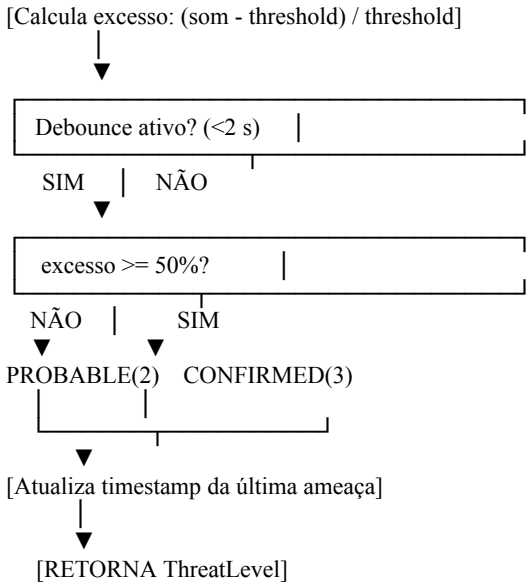
f) Fluxogramas

Algoritmo de Aprendizado Acústico



Detecção de Anomalias





g) Indicação do Uso de IA

Este projeto foi desenvolvido com auxílio do assistente Claude (Anthropic), versão Sonnet 4.6, acessado via claude.ai durante o período de desenvolvimento.

Componente	Contribuição da IA	Contribuição Humana
Concepção do projeto	30% — refinamento do escopo	70% — definição do problema real
Código-fonte	70% — módulos alarm.c, sensors.c, display.c	30% — integração, GPIO, debug
Driver SSD1306	90% — framebuffer completo e fontes	10% — ajuste de pinos GP14/GP15
Algoritmo de aprendizado	80% — outliers e estatística robusta	20% — calibração do microfone
Interrupção GPIO (Botão B)	100% — correção do desarme	0% — ausente no código original
Documentação técnica	80% — estrutura e texto base	20% — revisão e dados específicos
Teste e validação	0%	100% — responsabilidade exclusiva

Uso específico da IA neste projeto

- Geração do algoritmo de aprendizado acústico com estatísticas robustas (μ, σ, remoção de outliers)
- Criação completa do driver SSD1306 e das telas de UI
- Implementação da lógica de dual-core com mutex
- Identificação e correção da ausência de interrupção GPIO para desarme de emergência
- Geração deste relatório técnico com base nos arquivos do projeto

Em todos os casos, o estudante foi responsável por definir os requisitos, revisar o código gerado, integrá-lo ao sistema, validar o funcionamento em hardware real e ajustar os parâmetros para detecção específica de falhas em máquinas.

h) Conclusão

Resultados Alcançados

Objetivo	Status
Aprendizado acústico com threshold adaptativo (μ + N·σ) e remoção de outliers	✅ **Implementado e funcional**
Dois níveis de ameaça graduados (PROBABLE / CONFIRMED)	✅ **Implementado e funcional**
Arquitetura dual-core com mutex inter-core	✅ **Implementado e funcional**
Driver OLED SSD1306 com 5 telas de UI	✅ **Implementado e funcional**
Interrupção GPIO no Botão B (desarme de emergência)	✅ **Implementado e funcional**
Joystick com filtro de confirmação (3 leituras)	✅ **Implementado e funcional**
Buzzer com padrões sonoros graduados por nível de ameaça	✅ **Implementado e funcional**
Ajuste de sensibilidade (2σ / 3σ / 4σ) via interface	✅ **Implementado e funcional**
Compensação de temperatura	⚠️ **Estrutura pronta, não finalizada**
DMA para display	⚠️ **Estrutura pronta, não finalizada**

Dificuldades Enfrentadas

- **valores DC offset variam entre placas e com temperatura → calibração automática com 500 amostras em silêncio no boot**Calibração do microfone:
- **race conditions no acesso à SystemState e ADC → mutex_t com seções críticas (g_state_mutex, g_adc_mutex)**Sincronização dual-core:
- **ruídos transientes (batidas, partidas) disparavam alarme → remoção de 10% outliers na calibração + debounce de 2 s**Falsos positivos:
- **leituras instáveis sem filtro → confirmação com 3 leituras consecutivas**Interface com joystick:
- **atualização 30 Hz causava travamentos → framebuffer + envio em chunks**Display I2C lento:

Melhorias Futuras

- Compensação por temperatura: usar ADC4 interno para compensar deriva térmica do microfone analógico
- DMA para display: implementar transferência DMA real para descarregar o Core 1 durante envio do framebuffer
- Persistência na Flash: salvar assinatura acústica (μ, σ, threshold) para não recalibrar a cada boot
- Espectro de frequência: implementar FFT para detectar frequências específicas de falha (BPFO, BPFI, 1× RPM)
- Dashboard remoto: WiFi + MQTT para monitoramento remoto dos níveis de ruído e alertas

Considerações Finais

O projeto demonstrou com sucesso a aplicação de conceitos avançados de sistemas embarcados em uma plataforma real:

- Aprendizado estatístico embarcado: cálculo de média robusta e desvio padrão com remoção de outliers no RP2040
- Processamento multi-core: divisão clara entre Core 0 (detecção) e Core 1 (interface)
- Sistema de tempo real: múltiplas tarefas periódicas (10 Hz, 2 Hz, 30 Hz) coordenadas via mutex
- Interface rica: display OLED, LED RGB, buzzer estéreo, joystick analógico

O resultado é um sistema de monitoramento preditivo funcional, adaptativo (aprende qualquer ambiente), configurável (3 níveis de sensibilidade) e de baixo custo (apenas componentes da BitDogLab V7) — ideal para detecção precoce de falhas em máquinas rotativas.